

ECE 443/518 – Computer Cyber Security

Lecture 09 The RSA Cryptosystem

Professor Jia Wang
School of Engineering, Applied Science, and Health
Illinois Institute of Technology

September 16, 2026

Outline

Modular n -th Root

Public-Key Cryptography

RSA

Reading Assignment

- ▶ This lecture: UC 6,7, except 7.6
- ▶ Next lecture: UC 8.1,8.5,13.3.1

Outline

Modular n -th Root

Public-Key Cryptography

RSA

Solve Modular n -th Root for Prime p

- ▶ Solve $x^5 \equiv 2 \pmod{13}$.
 - ▶ $x^{10} \equiv 4 \pmod{13}$, $x^{15} \equiv 8 \pmod{13}$, $x^{25} \equiv 6 \pmod{13}$.
 - ▶ Fermat's Little Theorem: $x^{13} \equiv x \pmod{13}$
 - ▶ So $x^{25} \equiv x^{13}x^{12} \equiv xx^{12} \equiv x \pmod{13}$.
 - ▶ Solution: $x \equiv 6 \pmod{13}$
- ▶ How about $x^n \equiv a \pmod{p}$?
 - ▶ Assume $\gcd(n, p-1) = 1$.
 - ▶ No, you can't use this method if $n = 2$.
 - ▶ Solve $ny \equiv 1 \pmod{p-1}$ for y (via EEA).
 - ▶ Solution: $x \equiv a^y \pmod{p}$, or practically $x = a^y \bmod p$.
 - ▶ Check: $x^n \equiv a^{ny} \equiv a^{(ny) \bmod (p-1)} \equiv a \pmod{p}$.
- ▶ Time complexity
 - ▶ EEA takes $O(N^3)$ time.
 - ▶ $a^y \bmod p$ can be completed in $O(N^3)$ time. (How?)
 - ▶ Overall $O(N^3)$ time again!

Example

- ▶ Solve $x^5 \equiv 10 \pmod{17}$.
- ▶ Apply EEA to solve $5y \equiv 1 \pmod{16}$
 - ▶ $y \equiv 13 \pmod{16}$
- ▶ $x \equiv 10^{13} \pmod{17}$
 - ▶ Can we use a calculator to compute $10^{13} \bmod 17$?

Square-and-Multiply

- ▶ Compute $10^{13} \bmod 17$
- ▶ $10^{13} \equiv 10^8 \cdot 10^4 \cdot 10^1 \pmod{17}$
 - ▶ Since $13 = (1101)_2$
- ▶ Use square to calculate $10^2 \bmod 17$, $10^4 \bmod 17$, etc.
 - ▶ $10^2 \equiv 100 \equiv 15 \pmod{17}$
 - ▶ $10^4 \equiv 225 \equiv 4 \pmod{17}$
 - ▶ $10^8 \equiv 16 \pmod{17}$
- ▶ So $x \equiv 10^{13} \equiv 16 \cdot 4 \cdot 10 \equiv 11 \pmod{17}$
- ▶ Indeed, this algorithm computes $a^y \bmod p$ in $O(N^3)$ time.
 - ▶ $O(N)$ modular multiplications.

Square-and-Multiply by Hand

$$\begin{aligned}10^{13} &\equiv 10^{12} \cdot 10 \\&\equiv 100^6 \cdot 10 \equiv 15^6 \cdot 10 \\&\equiv 225^3 \cdot 10 \equiv 4^3 \cdot 10 \\&\equiv 4^2 \cdot 40 \equiv 4^2 \cdot 6 \\&\equiv 16 \cdot 6 \equiv 96 \equiv 11 \pmod{17}\end{aligned}$$

► Be creative with your calculators!

Modular n -th Root where m is not Prime

$$x^n \equiv a \pmod{m}.$$

- ▶ What if m is not a prime number?
- ▶ Consider $m = pq$ where $p \neq q$ are both prime numbers.
- ▶ Idea
 - ▶ Solve the equation for p and q individually.
 - ▶ Then combine the results.

Solve Modular n -th Root

$$x^n \equiv a \pmod{m}, \quad \text{where } m = pq.$$

- ▶ By Fermat's Little Theorem,
 - ▶ For $ny \equiv 1 \pmod{p-1}$, $(a^y)^n \equiv a \pmod{p}$.
 - ▶ For $ny' \equiv 1 \pmod{q-1}$, $(a^{y'})^n \equiv a \pmod{q}$.
- ▶ By Chinese Remainder Theorem,
 - ▶ If we can choose $y = y'$, then $(a^y)^n \equiv a \pmod{pq}$.
 - ▶ This is possible if $\gcd(n, (p-1)(q-1)) = 1$.
 - ▶ Solve $ny \equiv 1 \pmod{(p-1)(q-1)}$ to obtain y .
- ▶ We can solve $x^n \equiv a \pmod{pq}$ if $\gcd(n, (p-1)(q-1)) = 1$.
 - ▶ Solution: $x \equiv a^y \pmod{m}$, or practically $x = a^y \bmod m$.
 - ▶ Time complexity: $O(N^3)$
- ▶ Note that you cannot use this method to solve the seemingly very simple case of $x^2 \equiv a \pmod{pq}$.

Example

- ▶ Solve $x^5 \equiv 197 \pmod{221}$.
 - ▶ $221 = 13 * 17$
- ▶ Apply EEA to solve $5y \equiv 1 \pmod{192}$
 - ▶ $y \equiv 77 \pmod{192}$
- ▶ To compute $x \equiv 197^{77} \pmod{221}$ directly,
 - ▶
$$\begin{aligned}x &\equiv 197 \cdot 197^{76} \equiv 197 \cdot 134^{38} \equiv 197 \cdot 55^{19} \\&\equiv 197 \cdot 55 \cdot 55^{18} \equiv 6 \cdot 152^9 \equiv 6 \cdot 152 \cdot 152^8 \\&\equiv 28 \cdot 120^4 \equiv 28 \cdot 35^2 \equiv 45 \pmod{221}\end{aligned}$$
- ▶ Computer programs could use Chinese Remainder Theorem to compute x fast in practice.
 - ▶ $x \equiv 197^{77} \equiv 2^{77} \equiv 2^5 \equiv 6 \pmod{13}$
 - ▶ $x \equiv 197^{77} \equiv 10^{77} \equiv 10^{13} \equiv 11 \pmod{17}$
 - ▶ $x \equiv 45 \pmod{221}$

An Observation

$$x^n \equiv a \pmod{m}.$$

- ▶ But what if you don't know p and q for $m = pq$?
 - ▶ Factor m into pq first, or
 - ▶ Brute force: try $x = 1, 2, \dots, m - 1$
- ▶ What are their time complexities?
 - ▶ Any better algorithms?
- ▶ Is this observation of any practical importance?

Outline

Modular n -th Root

Public-Key Cryptography

RSA

Symmetric Cryptography Revisited

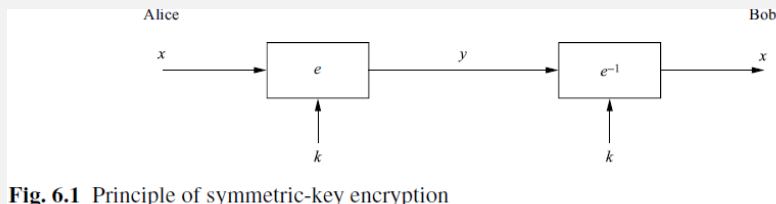


Fig. 6.1 Principle of symmetric-key encryption

(Paar and Pelzl)

- ▶ With the use of MAC as needed.
- ▶ Issue with Key Distribution: to establish a secret channel using symmetric cryptography, Alice and Bob need a secret channel to share the secret key k .
- ▶ Issue with Number of Keys: for a group of n people to communicate securely among each two of them, each people need to manage n keys and a total of $\frac{n(n-1)}{2}$ keys are needed.
- ▶ Issue with Nonrepudiation: Alice cannot prove to a third party that a ciphertext (with MAC) was sent by Bob as she also know the secret key k to generate the ciphertext.

Public-Key Cryptography

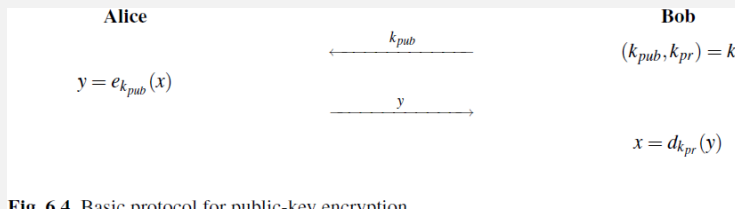


Fig. 6.4 Basic protocol for public-key encryption

- ▶ Key pair k : a public k_{pub} and a private (secret) k_{pr} .^(Paar and Pelzl)
 - ▶ No one should be able to derive k_{pr} from k_{pub} .
- ▶ Key Distribution: to establish a secret channel, Alice only need to obtain Bob's k_{pub} via an authentic channel.
- ▶ Number of Keys: each people just need to manage 1 key no matter how many people are there in the group.
- ▶ Nonrepudiation: via digital signatures if roles of k_{pr} and k_{pub} can be exchanged.
- ▶ Only if we could find such a cipher ...
 - ▶ For computationally unbounded adversaries?
 - ▶ For computationally bounded adversaries?

A Simple Hybrid Protocol

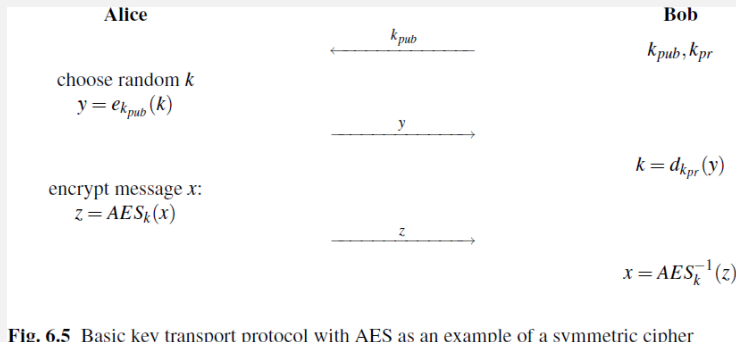


Fig. 6.5 Basic key transport protocol with AES as an example of a symmetric cipher

(Paar and Pelzl)

- ▶ In practice, symmetric ciphers remain very useful as public-key ciphers are usually orders of magnitude slower.
 - ▶ Use public-key ciphers to create a “slower” secure channel from an authentic channel between Alice and Bob.
 - ▶ Then Alice and Bob can use this “slower” secure channel to establish the secret key for symmetric ciphers, and thus create a “faster” secure channel.

Outline

Modular n -th Root

Public-Key Cryptography

RSA

History of RSA

- ▶ 1977: created by Ronald Rivest, Adi Shamir and Leonard Adleman
- ▶ 1983: RSA patent granted in US
- ▶ 1997: Clifford Cocks' equivalent system when working in the British intelligence agency GCHQ in 1973 was declassified.
- ▶ 2000: RSA patent expired in US

RSA Key Generation

- ▶ Choose two prime numbers p and q .
- ▶ Compute $n = pq$.
- ▶ Choose a positive integer e such that $\gcd(e, (p-1)(q-1)) = 1$.
- ▶ Solve $de \equiv 1 \pmod{(p-1)(q-1)}$ for a positive integer d .
- ▶ Public key: $k_{pub} = (n, e)$
- ▶ Private key: $k_{pr} = (p, q, d)$

RSA Encryption

- ▶ Public key: $k_{pub} = (n, e)$
- ▶ Plaintext: $x \in \{0, 1, \dots, n - 1\}$.
- ▶ Encryption: $y = e_{k_{pub}}(x) = x^e \bmod n$.
 - ▶ Ciphertext: $y \in \{0, 1, \dots, n - 1\}$.
- ▶ Example: $k_{pub} = (n = 221, e = 5)$
 - ▶ $x = 45, y = 45^5 \bmod 221 = 197$.

RSA Decryption

- ▶ Private key: $k_{pr} = (p, q, d)$
- ▶ Decryption: $x = d_{k_{pr}}(y) = y^d \bmod pq$.
- ▶ Example: $k_{pr} = (p = 13, q = 17, d = 77)$
 - ▶ $y = 197, x = 197^{77} \bmod 221 = 45$.
- ▶ Use a public key from *Bob*, Alice can only encrypt the message but cannot decrypt the message.
 - ▶ Why? What are our assumptions?

Oscar's Attacks

- ▶ Oscar knows $k_{pub} = (n, e)$ and the ciphertext y .
 - ▶ Assume n to be N bits.
- ▶ Apply brute force to find x
 - ▶ Need $O(2^N)$ time.
- ▶ Factor n into p and q
 - ▶ Apply integer factorization.
 - ▶ If p and q are chosen to be around $\frac{N}{2}$ -bit, then this will take Oscar $O(2^{\frac{N}{2}})$ time.
- ▶ Both are not practical for large N .
 - ▶ At least $N = 2048$ to be secure in long term.

Padding

- ▶ Oscar may derive useful statistics about plaintext from ciphertext since RSA is deterministic.
- ▶ Oscar may recover small x if e is small by trying to compute $\sqrt[e]{y}$, $\sqrt[e]{y+n}$, etc. using usual (non-modular) math.
- ▶ Oscar may modify y to change the plaintext in predictable ways: for any chosen s , if $y' = s^e y$, then $x' = d_{k_{pr}}(y') = sx$.
- ▶ Use padding to introduce random structure into plaintext.
- ▶ E.g. Optimal Asymmetric Encryption Padding (OAEP) in Public Key Cryptography Standard #1 (PKCS #1).
- ▶ A lot of other considerations for both security and performance.

Summary

- ▶ RSA
 - ▶ Key generation: by Bob, $k_{pub} = (n, e)$, $k_{pr} = (p, q, d)$
 - ▶ Encryption: everyone, $y = e_{k_{pub}}(x) = x^e \bmod n$.
 - ▶ Decryption: Bob only, $x = d_{k_{pr}}(y) = y^d \bmod pq$.
 - ▶ Assumption: Oscar cannot factorize n into p and q in polynomial time.
- ▶ Similar to other cryptosystems, there are a lot of pitfalls for actual implementation – you should follow documented standards exactly or use an established library instead.